
Emotion-IA

Reconnaissance Faciale d'Émotions en Temps Réel
et Optimisation Adaptative via Reinforcement Learning

Rapport de projet

Mouhamed DIOP
Abibatou WANDAOGO
Samba DIALLO

Résumé

Ce rapport présente le projet Emotion-IA, un système intelligent de reconnaissance d'expressions faciales combinant Deep Learning et Apprentissage par Renforcement. Le système utilise un réseau de neurones convolutifs (CNN) pour classifier sept émotions faciales à partir du dataset FER-2013, et un agent Deep Q-Network (DQN) pour optimiser dynamiquement les paramètres du flux vidéo afin de maximiser la confiance de classification.

Les résultats obtenus montrent une précision de classification de **65,8 %** sur le jeu de test, avec une amélioration de **+19,8 %** de la confiance apportée par l'agent DQN. Le système fonctionne en temps réel à **18–22 FPS**.

Mots-clés : *Deep Learning, Reinforcement Learning, DQN, Reconnaissance d'émotions, Computer Vision, CNN, Optimisation temps réel.*

Table de matières

1 Introduction

- 1.1 Contexte
- 1.2 Problématique
- 1.3 Solution proposée
- 1.4 Contribution

2 Méthodologie

- 2.1 Architecture globale
- 2.2 Dataset FER-2013
- 2.3 Classificateur CNN
- 2.4 Agent DQN

3 Résultat

- 3.1 Performance du classificateur CNN
- 3.2 Performance de l'agent DQN
- 3.3 Performance en temps réel
- 3.4 Robustesse au conditions variables

4 Discussion

- 4.1 Points forts
- 4.2 Limitations
- 4.3 Applications potentielles

5 Amélioration future

6 Conclusion

7 Annexe

8 Outils IBM Bob

1. Introduction

1.1 Contexte

La compréhension de l'état émotionnel humain représente un enjeu majeur pour l'évolution des interfaces numériques et l'interaction homme-machine. Les systèmes de reconnaissance d'émotions trouvent des applications dans de nombreux domaines : sécurité, santé mentale, marketing, éducation et interfaces utilisateurs adaptatifs.

1.2 Problématique

Les modèles de vision par ordinateur classiques souffrent souvent d'une perte de précision face à des environnements changeants. Les principales sources de dégradation sont :

- Les variations d'éclairage (faible luminosité, contre-jour) ;
- La qualité d'image variable (résolution, bruit) ;
- Les conditions de capture non contrôlées ;
- La diversité des caractéristiques faciales.

L'objectif est de créer un système auto-adaptatif capable de maintenir une haute fiabilité de détection malgré ces contraintes.

1.3 Solution proposée

Notre approche hybride combine deux paradigmes complémentaires de l'intelligence artificielle moderne. D'une part, un réseau CNN performant est entraîné sur le dataset FER-2013 pour classer sept émotions faciales. D'autre part, un agent DQN est chargé de réguler dynamiquement les paramètres du flux vidéo en temps réel.

1.4 Contributions

1. Une architecture CNN optimisée pour la reconnaissance d'émotions
2. Un environnement RL personnalisé pour l'ajustement de paramètres vidéo
3. Une intégration cohérente entre Deep Learning et Reinforcement Learning
4. Une interface web interactive (Streamlit) pour la démonstration en temps réel
5. Une documentation complète et un code source open-source.

2. Méthodologie

2.1 Architecture globale

Le système se compose de trois modules principaux intégrés dans un pipeline de traitement. La détection des visages est assurée par un classifieur Haar Cascade. Les visages détectés sont ensuite transmis au CNN pour la classification en sept classes d'émotions. Enfin, le score de confiance produit est exploité par l'agent DQN pour ajuster dynamiquement les paramètres du flux vidéo (contraste et exposition), formant ainsi une boucle de rétroaction en temps réel.

2.2 Dataset FER-2013

Le dataset FER-2013 regroupe 35 887 images en niveaux de gris de résolution 48×48 pixels, réparties en sept classes d'émotions : colère (angry), dégoût (disgust), peur (fear), joie (happy), tristesse (sad), surprise (surprise) et neutralité (neutral). La répartition standard distingue 28 709 images pour l'entraînement et 7 178 pour le test. La distribution entre classes est notablement déséquilibrée, la classe disgust étant particulièrement sous-représentée.

2.3 Classificateur CNN

2.3.1 Architecture du réseau

Le classificateur repose sur une architecture CNN à trois blocs convolutifs successifs, suivis d'une couche entièrement connectée. Chaque bloc suit le schéma : Conv2D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Dropout(0,25).

Le modèle totalise 620 935 paramètres, dont 620 007 entraînaibles et 928 gelés.

Couche	Filtres / Unités	Activation
Entrée ($48 \times 48 \times 1$)	—	—
Bloc Conv 1	32 filtres	ReLU
Bloc Conv 2	64 filtres	ReLU
Bloc Conv 3	128 filtres	ReLU
Dense	256 unités	ReLU
Sortie	7 classes	Softmax

Tableau — Architecture du réseau CNN

2.3.2 Techniques d'optimisation

Plusieurs stratégies ont été mises en place pour améliorer la généralisation du modèle :

- **Data Augmentation** : rotations ($\pm 15^\circ$), translations ($\pm 10\%$), zoom ($\pm 10\%$)
- **Batch Normalization** : normalisation des activations à chaque couche
- **Dropout** : taux de 0,25 (couches conv.) et 0,50 (couche dense)
- **Early Stopping** : arrêt si aucune amélioration sur 10 époque consécutive
- **Réduction du taux d'apprentissage** : facteur 0,5 en cas de plateau.

2.4 Agent DQN

2.4.1 Environnement de Reinforcement Learning

Espace d'états. L'état est un vecteur $s = [c, e, \text{conf}] \in \mathbb{R}^3$, où c représente le contraste, e l'exposition et conf la confiance de classification fournie par le CNN.

Fonction de récompense. L'agent maximise la confiance tout en pénalisant les paramètres hors plage acceptable : $r = \Delta \text{conf} \times 10,0 - 0,1 \times (1_{c(+)} + 1_{e(+)}).$ Les paramètres acceptables sont $c, e \in [0,7 ; 1,5]$.

Espace d'actions. L'espace comprend 9 actions discrètes formant une grille 3×3 , permettant d'ajuster le contraste et l'exposition de façon indépendante (diminuer / maintenir / augmenter).

2.4.2 Architecture DQN

Le réseau DQN est volontairement léger pour permettre une inférence en temps réel. Il est composé de quatre couches linéaires (Entrée(3) $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ Sortie(9)), avec des couches Dropout intercalées. Le modèle ne compte que 6 793 paramètres.

2.4.3 Hyperparamètres

Hyperparamètre	Valeur	Rôle
Facteur d'actualisation γ	0,95	Équilibre valeur future/immédiate
ϵ initial	1,0	Exploration initiale complète
ϵ minimal	0,01	Exploitation à terme
Décroissance ϵ	0,995	Transition douce vers l'exploitation

Taux d'apprentissage	0,001	Convergence stable
Taille du batch	32	Efficacité mémoire
Taille du replay buffer	2 000	Décorrélacion des expériences
Mise à jour réseau cible	10 épisodes	Stabilité de l'entraînement

Tableau — Hyperparamètres de l'agent DQN

3. Résultats

3.1 Performance du classificateur CNN

Métrique	Entraînement	Test
Accuracy	72,3 %	65,8 %
Loss	0,742	0,912
F1-Score (macro)	0,70	0,63

Tableau — Métriques de performance du CNN

L'analyse par classe révèle de fortes disparités liées au déséquilibre du dataset. La classe happy atteint 91,2 % de précision, tandis que la classe disgust, très minoritaire, plafonne à 50,7 %.

Émotion	Précision (%)
Happy	91,2 %
Surprise	85,8 %
Neutral	79,5 %
Angry	63,8 %
Sad	61,2 %
Fear	58,3 %
Disgust	50,7 %

Tableau — Précision par classe d'émotion

3.2 Performance de l'agent DQN

Métrique	Sans DQN	Avec DQN
Confiance moyenne	0,652	0,781
Amélioration	—	+19,8 %
Taux de succès	—	87,3 %
Temps par image	0,12 s	0,18 s

Tableau — Comparaison avec et sans optimisation DQN

L'agent DQN converge en environ 200 épisodes d'entraînement. La contrepartie de cette amélioration est une augmentation du temps de traitement de 50 % par image (de 0,12 s à 0,18 s), ce qui reste compatible avec les exigences temps réel.

3.3 Performance en temps réel

Métrique	Webcam	Fichier vidéo
FPS moyen	18,5	22,3
Latence (ms)	54	45
Utilisation CPU	45 %	38 %

Tableau — Métriques de performance en temps réel

3.4 Robustesse aux conditions variables

Condition	Amélioration (%)
Éclairage faible	+23 %
Éclairage fort	+18 %
Contre-jour	+12 %
Éclairage normal	+8 %

Tableau — Amélioration de la confiance selon les conditions d'éclairage

Le bénéfice de l'optimisation DQN est d'autant plus marqué que les conditions sont défavorables, ce qui valide l'intérêt de l'approche hybride.

4. Discussion

4.1 Points forts

Le système présente plusieurs qualités notables. L'amélioration de confiance de +19,8 % apportée par l'agent DQN est significative et mesurable. L'adaptation automatique aux conditions variables se révèle particulièrement efficace dans les situations difficiles (jusqu'à +23 % en faible lumière). La cadence de 18 à 22 FPS maintenue en continu garantit une utilisation fluide en temps réel.

4.2 Limitations

6. Le temps de traitement augmente de 50 % lorsque le DQN est activé.
7. Les performances sur la classe disgust restent faibles (50,7 %) en raison du fort déséquilibre du dataset.
8. La détection de visages multiples est traitée de façon séquentielle, ce qui peut limiter le débit sur des scènes chargées.
9. Seuls le contraste et l'exposition sont actuellement pilotés par l'agent ; d'autres paramètres vidéo pourraient être intégrés.

4.3 Applications potentielles

- **Sécurité** : surveillance adaptative en conditions d'éclairage variables ;
- **Santé** : monitoring émotionnel de patients en soins continus ;
- **Éducation** : mesure de l'engagement des étudiants en e-learning ;
- **Marketing** : analyse des réactions à des contenus ou produits ;
- **Automobile** : détection de la fatigue du conducteur.

5. Améliorations futures

Court terme.

Intégrer les variantes Double DQN et Dueling DQN, ainsi que le mécanisme de Prioritized Experience Replay, afin d'accélérer la convergence de l'agent et d'améliorer la stabilité de l'apprentissage.

Moyen terme.

Étendre le nombre de paramètres vidéo contrôlés (netteté, balance des blancs, etc.), traiter simultanément plusieurs visages dans un même flux, et intégrer des modèles de détection plus robustes comme MTCNN ou RetinaFace. Le recours au transfer learning permettrait également d'améliorer les performances sur les classes sous-représentées.

Long terme.

Explorer des algorithmes Actor-Critic plus avancés (PPO, SAC), des méthodes de méta-apprentissage pour la généralisation à de nouvelles conditions, ainsi que des architectures Vision Transformer pour le classifieur. Le déploiement sur des plateformes embarquées (edge computing) constitue également un objectif important pour les applications temps réel contraintes.

6. Conclusion

Ce projet a démontré la faisabilité d'une approche hybride combinant Deep Learning et Apprentissage par Renforcement pour la reconnaissance d'expressions faciales en temps réel. L'intégration d'un agent DQN comme module d'optimisation adaptative constitue une contribution originale qui améliore sensiblement la robustesse du système dans des conditions d'acquisition variables.

Les résultats obtenus : **65,8 % de précision CNN** sur FER-2013, **+19,8 % de gain de confiance** grâce au DQN, et un fonctionnement stable à **18–22 FPS** confirment la pertinence de l'approche. Les limitations identifiées dessinent une feuille de route claire pour les travaux futurs.

Le code source est disponible à l'adresse : <https://github.com/mouhamed-diop8/PROJECT-IA>

Références

- [1] *Mnih, V., et al.* (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- [2] *Van Hasselt, H., Guez, A., & Silver, D.* (2016). Deep reinforcement learning with double Q-learning. *AAAI Conference on Artificial Intelligence*.
- [3] *Wang, Z., et al.* (2016). Dueling network architectures for deep reinforcement learning. *ICML*.
- [4] *Goodfellow, I. J., et al.* (2013). Challenges in representation learning: FER-2013 dataset. *ICONIP*.
- [5] *LeCun, Y., Bengio, Y., & Hinton, G.* (2015). Deep learning. *Nature*, 521(7553), 436–444.

Annexe A — Commandes d'utilisation

A.1 Installation des dépendances

```
pip3 install opencv-python tensorflow pandas matplotlib seaborn scikit-learn pillow streamlit
```

A.2 Entraînement du classificateur CNN

```
python3 run_facial_recognition_simple.py
```

A.3 Entraînement de l'agent DQN

```
python3 train_dqn_system.py
```

A.4 Démonstration temps réel

```
python3 realtime_video_dqn.py --video 0
```

A.5 Interface web Streamlit

```
streamlit run streamlit_realtime_app.py
```

Annexe B — Structure du projet

```
PROJECT/ |-- run_facial_recognition_simple.py    # Entraînement CNN |--  
dqn_video_regulator.py                        # Module DQN |-- train_dqn_system.py  
# Entraînement DQN |-- realtime_video_dqn.py    # Traitement  
temps réel |-- streamlit_realtime_app.py        # Interface web |--  
best_model.h5                                # Modèle CNN entraîné |--  
dqn_agent.h5                                # Agent DQN entraîné |-- train/  
# Données d'entraînement | |-- angry/ disgust/ fear/ happy/ | |--  
sad/ surprise/ neutral/ |-- test/              # Données  
de test
```

Outils IBM Bob

L'outil Projet Bob d'IBM a été utilisé tout au long du développement du projet, principalement pour la génération de code, l'organisation de la structure du projet et l'assistance dans l'analyse des données. Son intégration dans Visual Studio Code a permis une interaction directe avec les fichiers du projet et un gain de temps important lors du développement.

Points forts

- Génération rapide et efficace de code de bonne qualité.
- Organisation automatique et structurée des dossiers et des fichiers du projet.
- Forte capacité à comprendre le contexte global du projet.
- Proposition d'améliorations pertinentes et d'idées avancées pour enrichir les analyses de données.
- Intégration directe avec l'environnement de développement, facilitant les modifications et les mises à jour du projet.
- Réduction significative du temps consacré aux tâches répétitives et à la rédaction de code.

Points faibles

- L'outil propose parfois des fonctionnalités ou des évolutions très ambitieuses qui semblent intéressantes et pertinentes au premier abord.
- Dans certains cas, il rencontre des difficultés à implémenter correctement ces fonctionnalités avancées.
- Lorsqu'il tente de corriger ces difficultés, il peut modifier ou supprimer des parties du code précédemment fonctionnelles.
- Une supervision humaine reste nécessaire pour valider les modifications proposées et garantir la stabilité du projet.

Bilan générale

L'utilisation de Projet Bob d'IBM a été très bénéfique pour le développement du projet grâce à ses capacités avancées de génération de code et d'analyse. Malgré certaines limites lors de l'implémentation de fonctionnalités complexes, l'outil constitue une aide précieuse qui permet d'accélérer le développement et d'explorer de nouvelles pistes d'amélioration.